RECRUITMENT ASSIGNMENT – BACKEND DEVELOPMENT

Task:

Build the backend for a "CodeChef Contest Control Center".

Problem Statement:

CodeChef VIT is organizing a large-scale competitive programming contest.

The organizing team requires a backend system capable of managing participants, problems, submissions, leaderboards, contest states, and contest operations.

Your task is to design and implement the backend services powering the Contest Control Center.

The system should focus on scalability, clean architecture, data consistency, and business logic implementation rather than simple CRUD operations.

Tech Stack:

Choose Any One:

- Node.js + Express.js
- GoLang / Nest.js
- FastAPI/Flask/Django

Database:

Choose Any One:

- PostgreSQL / MySQL(Preferred)
- MongoDB

Core Functionalities

1. Contest Management

The system should support:

- Create Contest
- Get Contest Details
- Update Contest
- Start Contest
- End Contest

Contest should contain:

- Name
- Description
- Start Time
- End Time
- Contest Status

Statuses:

• Draft
• Scheduled
• Live
• Ended

Business Rules:

• A contest cannot be started twice.
• End Time must always be greater than Start Time.
• Contest status transitions must be valid.

2. Problem Management

Each contest contains multiple problems.

Problem Fields:

• Title
• Difficulty
• Points
• Time Limit
• Memory Limit

Features:

• Create Problem
• Update Problem
• Delete Problem
• Get Problem Details

Business Rules:

• Problem titles must be unique within a contest.
• Points cannot be negative.

3. Participant Registration

Participants should be able to register for a contest.

Features:

• Register Participant
• Get Participant Details
• Get Registered Participants

Business Rules:

• Duplicate registrations are not allowed.
• Registration closes once the contest becomes Live.

4. Submission System

Participants can submit solutions.

Submission Fields:

- Participant
- Problem
- Language
- Submission Time
- Verdict

Verdicts:

- Pending
- Running
- Accepted
- Wrong Answer
- Runtime Error
- Time Limit Exceeded

Features:

- Create Submission
- Get Submission
- Get Participant Submissions
- Get Problem Submissions

Business Rules:

- Submissions cannot be made after the contest ends.
- Submission must belong to a registered participant.
- Submission must belong to an existing contest problem.

5. Dynamic Leaderboard

Implement automatic leaderboard generation.

Leaderboard Ranking Rules:

1. Higher solved count ranks higher.

2. If solved count is equal:
   Lower penalty time ranks higher.

Leaderboard APIs:

- Get Leaderboard
- Get Participant Rank

Leaderboard should be generated dynamically from submission data.

No leaderboard table should be manually maintained.

6. Freeze Mode

The contest may enter Freeze Mode during the final phase.

Features:

• Enable Freeze Mode
• Disable Freeze Mode

Business Rules:

• Submissions continue to be accepted.
• Leaderboard must stop updating.
• Internal ranking calculations continue.
• Once Freeze Mode is disabled:
Leaderboard should automatically reflect all hidden changes.

7. Rejudge System

Organizers may rejudge submissions.

Example:

Accepted → Wrong Answer

Wrong Answer → Accepted

Features:

• Rejudge Submission

Business Rules:

• Leaderboard must automatically recalculate.
• Participant statistics must update automatically.
• Rejudge history must be preserved.

8. Activity Logging

Every important system action must be logged.

Examples:

• Contest Created
• Contest Started
• Participant Registered
• Submission Created
• Submission Rejudged
• Freeze Mode Enabled
• Freeze Mode Disabled

Activity APIs:

• Get Activities
• Get Activity By ID

Authentication & Authorization

Implement JWT Authentication.

Roles:

• Admin
• Organizer
• Participant

Permissions:

Admin

• Full Access

Organizer

• Manage Contest
• Manage Problems
• Rejudge Submissions
• View Analytics

Participant

• Register
• Submit Solutions
• View Leaderboard

Role-based access control must be implemented.

Expected Features

• Input Validation
• Pagination
• Filtering
• Sorting
• Error Handling
• Environment Variables
• Modular Folder Structure
• Database Integration
• Authentication
• Authorization

Suggested APIs

AUTH

POST /auth/register
POST /auth/login

CONTEST

POST /contests
GET /contests
GET /contests/
PATCH /contests/

PROBLEMS

POST /problems
GET /problems
GET /problems/
PATCH /problems/

PARTICIPANTS

POST /participants/register
GET /participants

SUBMISSIONS

POST /submissions
GET /submissions
GET /submissions/

LEADERBOARD

GET /leaderboard
GET /leaderboard/rank/

FREEZE MODE

POST /freeze/enable
POST /freeze/disable

REJUDGE

POST /submissions/rejudge

ACTIVITIES

GET /activities

README MUST CONTAIN

• Project Setup Instructions
• Tech Stack Used
• Database Schema
• Entity Relationship Diagram
• API Documentation
• Request & Response Examples
• Assumptions Made
• Deployment Link (if deployed)

Evaluation Criteria

• Database Design
• API Design
• Business Logic Implementation
• Authentication & Authorization
• Clean Architecture
• Error Handling
• Documentation Quality
• Scalability
• Code Quality
• Deployment

Important Note:

**You may take help from AI tools. However, you should be able to explain every architectural, database, and implementation decision made in your project. Candidates may be cross-examined during interviews regarding their logic, design choices, and understanding of the system. **
Minimum 60% of the features must be included for advancing to the interview round.